MAANG TypeScript Interview Guide

ADVANCED TYPE SYSTEM MECHANICS, RUNTIME BEHAVIOR & ENGINEERING PATTERNS

PART I: CORE FUNDAMENTALS & TYPE ENGINE MECHANICS

Questions 01 - 05

1. What is TypeScript, and how does it differ from JavaScript?

TypeScript is a typed, syntactic superset of JavaScript developed by Microsoft that adds static type-checking at compile time. While JavaScript is dynamically and weakly typed (types evaluated at runtime), TypeScript introduces a static type system that enables early error detection, IDE auto-completion, and refactoring safety. TypeScript code cannot execute directly in engines like V8; it must undergo **transpilation** to JavaScript, stripping out all types, interfaces, and generic annotations.

2. How does TypeScript compilation work, and what happens to types at runtime?

The TypeScript compiler (`tsc`) operates in two distinct phases:
1. **Type Checking:** Parses source code into an AST (Abstract Syntax Tree), performs type inference, and reports static type errors.
2. **Code Generation (Type Erasure):** Strips all type annotations, interfaces, type aliases, and generics, emitting plain JavaScript code targetable to specific ECMAScript versions (e.g., ES6, ESNext).
Runtime Impact: Types do *not* exist at runtime. TypeScript types do not affect memory allocation, performance, or runtime validation. All type checking is strictly compile-time.

3. What is the difference between type inference and explicit type annotations?

Type Inference occurs when the TypeScript compiler automatically determines the type of an expression based on initial values, return statements, or contextual usage (e.g., `let x = 10` infers `number`). **Explicit Type Annotations** manually specify types using syntax like `let x: number = 10`. *MAANG Insight:* Prefer type inference for local variable initialization to keep code concise, but use explicit annotations for function signatures, public API contracts, and complex data structures to prevent unintended type drift.

4. What is the difference between any, unknown, and never?

- `any`: Disables type checking entirely. Allows assigning any value and calling any method without checking. It bypasses sound safety.
- `unknown`: The type-safe counterpart to `any`. Represents any value, but TypeScript forbids performing arbitrary operations or calling methods on an `unknown` variable without prior **type narrowing** or assertion.
- `never`: The bottom type representing values that never occur. Used for functions that never return (e.g., throw errors, infinite loops) or exhaustive check branches in union narrowing.

5. What is the difference between void and never?

`void` indicates that a function completes execution normally but returns no meaningful value (implicitly or explicitly returning `undefined`). In contrast, `never` indicates that a function **never completes execution or returns at all**—either because it unconditionally throws an exception or enters an infinite execution loop. `void` is a unit-like type, whereas `never` is an empty set (bottom type).

PART II: TYPE ALGEBRA, NARROWING & STRUCTURAL TYPING

Questions 06 - 10

6. What is the difference between union types and intersection types?

• **Union Types (`A | B`):** Represents a value that can be *either* type `A` *or* type `B`. Set-theoretically, it is the **union** of allowed values. Accessing properties on a union only permits properties common to both types unless narrowed.
• **Intersection Types (`A & B`):** Combines multiple types into one, requiring the value to possess *all* properties of both `A` *and* `B` simultaneously (set intersection of types, set union of properties).

7. What are literal types, and how do they differ from ordinary types?

Ordinary primitive types (e.g., `string`, `number`, `boolean`) represent an entire domain of infinitely many possible values. **Literal Types** represent specific, exact primitive values (e.g., `"GET"`, `404`, `true`). Literal types allow developers to create strict value sets using unions (e.g., `type HttpMethod = "GET" | "POST" | "DELETE"`), allowing static catching of typos or unsupported values.

8. What is type narrowing, and how do type guards work?

Type Narrowing is the process where TypeScript refines a broad type (like a union) into a more specific type within a conditional block. **Type Guards** are runtime expressions used to trigger narrowing:
1. Built-in runtime checks: `typeof x === "string"`, `x instanceof Date`, `"key" in obj`.
2. User-defined Type Predicates: Functions returning `val is TargetType` (e.g., `function isUser(x: any): x is User { return "name" in x; }`).
3. Discriminated properties checking literal properties.

9. What is the difference between type and interface?

While both define object contracts, key differences include:
• **Declaration Merging:** Multiple `interface` definitions with the same name automatically merge their fields. `type` aliases throw duplicate identifier errors.
• **Primitives & Unions:** `type` can alias primitives, tuples, and unions (e.g., `type ID = string | number`). `interface` can only model object shapes and functions.
• **Extending:** Interfaces extend via `extends`; type aliases combine via intersection (`&`).
Rule of thumb: Use `interface` for OOP domain modeling and public API boundaries; use `type` for complex type compositions, unions, and tuples.

10. What is structural typing, and how does TypeScript determine type compatibility?

TypeScript uses a **Structural Type System** (also called duck typing) rather than a Nominal Type System (like Java/C#). Type compatibility is determined purely by the *shape and properties* of an object, not its explicit declaration or inheritance hierarchy. If type `A` has all required properties of type `B` with compatible types, `A` is assignable to `B`, even if `A` contains additional properties.

PART III: OBJECT MODIFIERS & GENERIC METAPROGRAMMING

Questions 11 - 15

11. Optional properties vs nullable types vs optional parameters

- **Optional Property (`key?: T`):** Indicates property `key` may be omitted on the object; its type is implicitly `T | undefined`.
- **Nullable Type (`T | null | undefined`):** Explicitly permits `null` or `undefined` values, regardless of key presence.
- **Optional Parameter (`fn(x?: T)`):** Allows passing `undefined` or omitting the argument during function invocation.

12. What is the difference between readonly properties and const variables?

- `const` applies to **variable bindings**. It prevents reassigning the variable reference itself, but properties of mutable objects declared with `const` can still be mutated.
- `readonly` is a type-system modifier for **object properties or arrays**. It prevents mutating the specific property (e.g., `obj.prop = 5` fails at compile time), though the underlying reference assigned to the variable could change if reassigned.

13. What are generics, and how do generic constraints work?

Generics enable building parameterized, reusable code components that work over a variety of types rather than a single static type. **Generic Constraints** limit the permissible types passed into a generic parameter using the `extends` keyword (e.g., `<T extends { id: string }>`). This guarantees that the generic type `T` possesses specific minimal properties inside the implementation.

14. How do you use keyof, typeof, and indexed access types?

- `keyof T`: Takes an object type and produces a string/number/symbol literal union of its keys (e.g., `keyof User` produces `"id" | "name"`).
- `typeof expr`: In type context, queries the TypeScript type of an existing variable/object (e.g., `type Config = typeof defaultConfig`).
- **Indexed Access Types (`T[K]`):** Looks up the specific type of property `K` on type `T` (e.g., `User["age"]` evaluates to `number`).

15. What are mapped types, and how would you create a custom mapped type?

Mapped Types build new types by iterating over keys in a union (typically created via `keyof`).
Syntax pattern: `type MyMapped<T> = { [K in keyof T]: NewType }`.
Modifiers can add/remove optionality or readonly flags using `+` or `-` (e.g., `-readonly` or `+?`).

```
type Nullable<T> = {
  [K in keyof T]: T[K] | null;
};
```

PART IV: ADVANCED METAPROGRAMMING & DISCRIMINATED UNIONS

Questions 16 - 20

16. What are conditional types, and how does the infer keyword work?

Conditional Types evaluate type choices based on a subtype relationship test: `T extends U ? X : Y`. The `infer` keyword is used inside the `extends` clause of a conditional type to introduce a type variable that is automatically extracted and pattern-matched from another structure.

```
type UnpackArray<T> = T extends (infer U)[] ? U : T;
```

17. Built-in Utility Types: Partial, Required, Pick, Omit, Record, ReturnType

- `Partial<T>`: Makes all properties in `T` optional.
- `Required<T>`: Makes all properties in `T` required.
- `Pick<T, K>`: Constructs a type by picking specific keys `K` from `T`.
- `Omit<T, K>`: Constructs a type by removing keys `K` from `T`.
- `Record<K, T>`: Maps object keys `K` to values of type `T`.
- `ReturnType<F>`: Extracts the return type of function type `F` using conditional `infer`.

18. Overload signatures vs union types in function definitions

- **Union Types in Parameters:** Suitable when function logic handles inputs uniformally or independently (e.g., `fn(x: string | number)`).
- **Function Overloads:** Define multiple function header signatures before a single implementation signature. Essential when the *return type strictly depends on the specific parameter type passed* (e.g., passing a string yields a string, while passing a number yields a number).

19. What are discriminated unions, and how do they enable exhaustive checks?

A **Discriminated Union** is a union of object types where each object contains a common, shared literal field (the "discriminant" tag, e.g., `kind: "circle" | "square"`). TypeScript uses this property to narrow types inside `switch` or `if` statements. Exhaustive type checking is enforced by assigning the remaining unhandled union case in the `default` branch to a `never` type variable: `const _exhaustiveCheck: never = shape;`.

20. Type assertions vs type casting, and why assertions can be unsafe

- **Type Assertion (`x as T` or `<T>x`):** Directs the compiler to override its inferred type and treat the value as type `T`. It creates **zero runtime JavaScript code or casting**.
- **Unsafety:** Assertions lie to the type checker. If a developer asserts `x as User` when `x` is actually an empty object, calling `x.name.toUpperCase()` passes compilation cleanly but explodes with a runtime `TypeError`.

PART V: SYSTEM ARCHITECTURE, VARIANCE & MODULES

Questions 21 - 25

21. What is the difference between enum, const enum, and union literal types?

• `enum`: Emits a real JavaScript object at runtime with bi-directional key-value mappings.
• `const enum`: Stripped at compile-time; usage sites have literal values inlined directly into emitted JS code.
• **Union Literal Types (`"A" | "B"`):** Preferred in modern TS. Completely erased at compile time, flexible with string inputs, zero bundle footprint.

22. What is the difference between as const and the satisfies operator?

• `as const` (Const Assertion): Deeply converts an object/array literal into `readonly` with literal primitive types (prevents type widening).
• `satisfies` (TS 4.9+): Validates that an expression matches a given interface/type **without widening or modifying the inferred type**. It preserves exact literal inferences while enforcing structural correctness.

23. How does TypeScript handle parameter variance and return type compatibility?

• **Return Types are Covariant:** A function returning a subtype (e.g., `Dog`) can be assigned to a function expecting a supertype (e.g., `Animal`).
• **Parameters are Contravariant:** Under `strictFunctionTypes: true`, function parameters are checked contravariantly (expecting broader types is safe). Under default methods in interfaces, they are bivariant for legacy compatibility.

24. Declaration files (.d.ts) and typing untyped JS libraries

`.d.ts` files contain ambient type declarations without implementation logic. They describe JavaScript libraries to TypeScript. To type an untyped third-party library: 1. Install DefinitelyTyped types: `npm i -D @types/library-name`. 2. Declare a custom module in a local `.d.ts` file: `declare module "untyped-lib" { export function doWork(): void; }`.

25. ES Modules vs CommonJS and moduleResolution in tsconfig.json

• `module`: Defines emitted JS import/export syntax (e.g., `CommonJS` with `require()` or `ESNext` with `import`).
• `moduleResolution`: Strategy used by compiler to resolve import paths:
– `Node` (legacy Node.js algorithm).
– `Node16` / `NodeNext`: Strict ESM/CJS interop respecting `package.json` `"type": "module"` and explicit file extensions.
– `Bundler`: Mode designed for Vite/Webpack/esbuild.

PART VI: ENTERPRISE PATTERNS, SAFETY & API DESIGN

Questions 26 - 30

26. Purpose of strict, strictNullChecks, and noImplicitAny

• `"strict": true`: Enables a family of strict type-checking compiler flags.
• `strictNullChecks`: Prevents `null` and `undefined` from being implicitly assigned to every type. Forces developers to explicitly handle non-existent values.
• `noImplicitAny`: Raises errors on expressions and parameters where TypeScript cannot infer a type and defaults to `any`.

27. Difference between object, {}, and Record<string, unknown>

• `object`: Represents any non-primitive type (excludes `number`, `string`, `boolean`, `symbol`, `null`, `undefined`).
• `{}` (Empty Object Type): Matches any value that is not `null` or `undefined` (including primitive numbers/strings).
• `Record<string, unknown>`: Represents an object with string keys and safe `unknown` values. Best choice for generic key-value dictionaries.

28. Safely typing API responses and validating external data at runtime

Compile-time TypeScript assertions on API data (e.g., `fetch().then(r => r.json() as User)`) are unsafe because external API contracts can drift or fail at runtime. **Solution:** Parse don't validate. Use runtime validation libraries (e.g., `Zod` or `TypeBox`) to validate JSON at boundary entry points and infer static TypeScript types directly from schemas.

29. Modeling complex state transitions using discriminated unions

Instead of modeling state with optional flags (e.g., `{ isLoading: boolean, data?: Data, error?: Error }` which allows invalid states like `isLoading: true` AND `data: Data`), use a Discriminated Union:

```
type State<T> =
  | { status: "idle" }
  | { status: "loading" }
  | { status: "success"; data: T }
  | { status: "error"; error: Error };
```

This makes invalid states unrepresentable in the type system.

30. Designing a type-safe, reusable API client

Leverage generics, mapped types, and infer to build strongly typed client wrappers:

```
async function apiFetch<TResponse>(
  url: string, options?: RequestInit
): Promise<{ data: TResponse | null; error: string | null }> {
  try {
    const res = await fetch(url, options);
    const data = await res.json();
    return { data: data as TResponse, error: null };
  } catch (e) {
    return { data: null, error: (e as Error).message };
  }
}
```

PART VII: OUTPUT-BASED & EXECUTION QUESTIONS

Questions 31 - 35

31. What is the output, and why?

```
type A = string | number;
function print(value: A) {
  if (typeof value === "string") {
    console.log(value.toUpperCase());
  } else {
    console.log(value * 2);
  }
}
print("hello");
print(10);
```

Console Output: `HELLO` then `20`
Explanation: `typeof value === "string"` acts as a type guard. During `print("hello")`, TypeScript narrows `value` to `string`, executing `toUpperCase()`. During `print(10)`, execution enters the `else` branch where `value` is narrowed to `number`, computing `10 * 2 = 20`.

32. Does this code compile, and what is the output if it does?

```
type User = { name: string; };
const user: User = { name: "Alex", age: 25 };
console.log(user);
```

Compilation Result: DOES NOT COMPILE
Explanation: TypeScript performs **Excess Property Checking** when an object literal is assigned directly to a variable with an explicit type annotation. Because `age` is not declared on type `User`, the compiler throws: `Type '{ name: string; age: number; }' is not assignable to type 'User'. Object literal may only specify known properties, and 'age' does not exist in type 'User'.`

33. What is the output, and what type does result have?

```
function identity<T>(value: T): T { return value; }
const result = identity("TypeScript");
console.log(result);
```

Console Output: `TypeScript`
Inferred Type: `result: string` (or string literal type `"TypeScript"`)
Explanation: The generic function `identity` infers parameter type `T` from the argument `"TypeScript"`. The return type matches `T`, making `result` strongly typed as a string.

34. What is the output, and why does TypeScript infer this type?

```
const status = "success" as const;
function checkStatus(value: "success" | "error") { console.log(value); }
checkStatus(status);
console.log(typeof status);
```

Console Output: `success` then `string`
Explanation: `as const` narrows the static type of `status` to the literal type `"success"` (satisfying `checkStatus`). However, JavaScript's runtime operator `typeof status` evaluates the underlying JS primitive value at execution time, returning `"string"`.

35. Does this code compile, and what happens when condition is evaluated?

```
type Result = | { status: "success"; data: string } | { status: "error"; message: string };
function handleResult(result: Result) {
  if (result.status === "success") { console.log(result.data); }
  else { console.log(result.message); }
}
handleResult({ status: "success", data: "Done" });
```